



Manual Técnico – Sistema ORBYT

Arquitetura • Tecnologias • Configuração • Banco de Dados • Melhorias Futuras

Equipe de Desenvolvimento

Alex Alves

github.com/alexx-al3

Andrey Gabriel

github.com/Andrey-Gabriel-Ben

Orientador: Prof. Rodrigo Costa

github.com/razevedocosta

1. Visão Geral

O ORBYT é uma aplicação desktop em **Java 20** com arquitetura orientada a objetos em camadas, destinada ao gerenciamento de clientes, equipamentos, usuários e ordens de serviço para a **Senac Solutions**.

2. Tecnologias Utilizadas

Tecnologia	Ferramenta / Lib	Finalidade
Linguagem	Java 20	Programação Orientada a Objetos
Interface Gráfica	Java Swing + FlatLaf	Look-and-feel moderno; validação visual de erros
Banco de Dados	PostgreSQL (Supabase)	Banco relacional gerenciado na nuvem
Conexão BD	JDBC – org.postgresql:postgresql	Acesso ao banco via Maven
Segurança	BCrypt – org.mindrot:jbcrypt	Hash de senhas dos usuários
Build	Maven (pom.xml)	Gerenciamento de dependências e compilação
Controle de Versão	Git + GitHub	Versionamento e colaboração da equipe

3. Arquitetura em Camadas

Camada	Pacote	Responsabilidade
Model	com.senacsolutions.model	Entidades: Cliente, Equipamento, OrdemServico, Usuario, Pagamento, StatusOS
DAO	com.senacsolutions.dao	Acesso ao banco via JDBC; CRUD e consultas SQL
Controller	com.senacsolutions.controller	Regras de negócio, validações, orquestração View ↔ DAO
View	com.senacsolutions.view	Telas Swing, captura de input, exibição de resultados e feedbacks visuais

4. Regras de Negócio Implementadas

Código	Regra	Implementação
RN01	CPF único	ClienteDao.buscarPorCpf() antes do INSERT + UNIQUE no banco
RN02	Número de série único	UNIQUE constraint em equipamento.num_serie
RN03	Prazo de 15 dias	Calendar.add(DAY_OF_MONTH, 15) no OrdemServicoController
RN04	Cores de prioridade	OrdemServico.getDiasRestantes() + RendererDePrioridade nas telas

Código	Regra	Implementação
RN05	Fluxo de status	Cada transição controlada por método específico do DAO
RN06	Custo obrigatório	Validação no Controller antes do INSERT
RN07	Descrição obrigatória	Campo NOT NULL no banco + validação no Controller
RN08	Técnico exclusivo	WHERE id_tecnico IS NULL no UPDATE de assumirOS()
RN09	Roteamento por perfil	switch(cargo) no UsuarioControler.efetuarLogin()
RN10	Senha mínima	senha.length() >= 8 validado no Controller

5. Configuração e Instalação

Pré-requisitos:

- Java 20 (JDK)
- Maven 3.x
- Conta no Supabase (PostgreSQL)
- Git

Passos:

- Clone o repositório: **git clone https://github.com/[usuario]/orbyt**
- Execute o script **Banco de dados/CriacaoBanco.sql** no Supabase
- Copie **Banco.properties.example** → **Banco.properties** e preencha as credenciais
- Execute: **mvn clean compile**
- Inicie o sistema: **mvn exec:java** (ou rode TelaLogin como classe principal)

6. Melhorias Futuras

- Implementação completa do módulo de Pagamento (PagamentoDao + PagamentoControler)
- Relatórios exportáveis em PDF (clientes, equipamentos, OS por período)
- Dashboard com gráficos de produtividade dos técnicos
- Notificação interna ao atendente quando OS é finalizada pelo técnico
- API REST para integração com interface web ou mobile
- Edição e exclusão de clientes/equipamentos com controle de integridade referencial
- Sistema de permissões granular (admin / atendente / técnico)
- Versão SaaS multi-empresa